determine whether a general directed graph contains a Hamiltonian path, what would be your solution?

(7) We all know what a minimum spanning tree is. Now you are asked to find the maximum spanning tree of an undirected graph. The maximum spanning tree is the spanning tree with the largest total weight. What is the complexity of your algorithm? Can this be solved in polynomial time?

(8) Your friend is organising a party for her classmates. There are N people in her class, excluding your friend. She also knows which pairs of her classmates know each other. She wants to invite as many people to the party as possible, subject to the following two constraints: (a) Every person invited should know at least 5 other people in the party; and (b) there should be 5 other people in the party whom a person invited to the party does not know. Write an algorithm that takes the list of class mates and the list of pairs who know each other, and outputs the best choice for the party's list of invitees. What is the complexity of your algorithm?

(9) You are given a sequence of 'n' numbers *a1, a2, …..* *aN*. A subsequence is a subset of these numbers taken in order of the form ai1, *ai2, ……… aiK*, where *1 ≤ i1 ≤ i2 ≤…. ik ≤ n*. An increasing subsequence is one whose elements are strictly in increasing order, such that ai1 ≤ *ai2 ≤…….. aiK*. You are asked to write a program to find the longest increasing subsequence given a sequence of *N* integers. For example, if you are given 5, 2, 8, 6, 3, 6, 9, 7, then the longest increasing subsequence is 2, 3, 6, 9. What is the complexity of your algorithm?

(10) You are given a string *'s'* of *N* characters, which you believe to be the garbled text of a document. However, the garbling has resulted in all punctuation of the original text getting removed. You are also given a dictionary, which can tell whether a particular sub-string is a word. You can check whether a string is a valid word in the dictionary by calling the function *'dict'* with a string; it returns *true* if the given string is a valid word in the dictionary, else it returns *false.* Write an algorithm to check whether the given sentence can be reconstructed into a valid sequence of words in the dictionary. Is it possible to write a greedy algorithm for solving this problem?

(11) Given a graph *G(V, E)* where *V* is the set of vertices, and *E* is the set of edges in the form of an adjacency matrix, a subset *S* of *V* is an independent set if there are no edges among the vertices in *S*. Is it possible to write a polynomial time algorithm to determine the largest independent set in a given directed graph? Now instead of a directed graph, you are given a tree. Can you write an algorithm for determining the largest independent set in the tree?

(12) A vertex cover of a graph *G(V, E)* is a subset of vertices *V' c V* such that each edge E is incident on at least one vertex in *V'*. That is, each edge *E* is covered by at least one vertex in *V'*, hence the name 'vertex cover'. If *G* is a tree instead of an arbitrary graph, write an algorithm to determine the minimum-size vertex cover of *G*.

(13) You are given a directed graph *G(V, E)* with positive edge weights. You are asked to write an algorithm that returns the length of the shortest cycle in the graph. Your algorithm should return 0 if the graph is acyclic. What is the complexity of your algorithm?

(14) You are given a directed acyclic graph *G(V, E)* and two vertices in the DAG, namely *x* and *y*. Write an algorithm to count the number of distinct paths in DAG between *x* and *y*.

(15) You are given a string *'s'* of length *N*. You are asked to write an algorithm to output all the permutations of the given string. What is the complexity of your algorithm?

My 'must-read book' for this month

This month's must-read book suggestion comes from our reader Nethravathi. She suggests a classic computer science book: 'A Discipline of Programming' by Edsger W Dijkstra. Nethravathi says that "…this is an essential book to understand programming from the mathematical formalism perspective. It builds a framework for building 'correct' programs and how to reason about 'correct' programs. While this is not an easy book to read, I would suggest it to anyone who is seriously interested in programming formally". Thank you, Nethravathi, for your suggestion!

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming—and here's wishing you a very happy new year in advance! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a researcher in IBM India Research Labs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' *at http://www.linkedin.com/groups?home=&gid=2339182.*